

Agent Synchronization Implementation Playbook

Typed coordination, tool authorization, deadlines, joins, and commits

CONDUCTOR | SPECIALISTS | JOIN | GATEKEEPER

1 Plan

required and optional roles

2 Run

parallel typed tasks

3 Commit

one validated decision

1. What is being synchronized?

Implementation status

This playbook is the synchronization design to implement. It does not claim that the current backend already runs these agents. The first deliverable is a deterministic synchronous pipeline; optional model-generated explanation comes later.

Agent synchronization here means coordinating bounded responsibilities under one authenticated request. It is not a group chat between models. Each role receives an immutable context, returns a typed result, and has no authority to mutate another role's output.

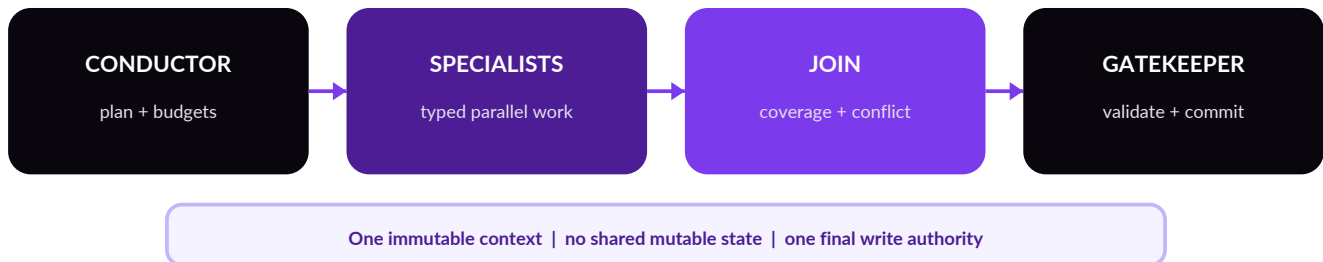


Figure 1. The only supported coordination path. Specialists do not call one another and do not commit the final decision.

1.1 Non-negotiable rules

- The authenticated API creates identity, tenant, purpose, consent pointer, deadline, budgets, and configuration versions.
- The Conductor plans and dispatches work but cannot invent product facts, permissions, or business policy.
- Every specialist has an immutable tool allowlist and returns an `AgentResult<T>` object.
- Missing required safety work produces NOT_SURE or BLOCKED; missing optional work is omitted.
- Referee applies policy, Skeptic calculates confidence, and Gatekeeper owns the only final commit path.
- No shared mutable JSON, cross-agent prompt memory, recursive agent calls, or raw database access from agents.

1.2 Request-level ownership

Stage	Owner	Output	May write?
Authenticate	Elysia middleware / Bodyguard	Trusted execution identity	Audit only
Plan	Conductor	Required/optional task plan	Plan audit
Execute	Specialists	Typed claims and evidence	No decision writes
Join	Conductor + Referee + Skeptic	Validated draft and confidence	Disagreement record only
Finalize	Gatekeeper	ProductDecision	Decision transaction

2. Shared contracts

2.1 Immutable execution context

Create this object after authentication and tenant resolution. Never accept its protected fields from model output or arbitrary request text.

```
type ExecutionContext = {
  requestId: string; traceId: string;
  tenantId: number; userId: number;
  purpose: 'product_decision'; locale: string;
  consentSnapshotId?: string;
  deadlineAt: number;
  budget: { toolCalls: number; tokens: number; costUnits: number };
  allowedToolsByRole: Record<AgentRole, readonly ToolId[]>;
  configVersions: { routing: string; policy: string; fit: string; prompt?: string };
}
```

2.2 Typed specialist result

```
type AgentResult<T> = {
  role: AgentRole;
  status: 'success' | 'partial' | 'not_sure' | 'blocked' |
  'retryable_failure' | 'permanent_failure';
  value: T | null;
  claims: Claim[]; evidence: EvidenceRef[]; warnings: Warning[];
  fallbackUsed?: string; startedAt: string; finishedAt: string;
  configVersions: Record<string, string>;
}
```

2.3 Required and optional roles

Condition	Required	Optional	Missing behavior
Every decision	Bodyguard, Detective, Boycott, Referee, Skeptic, Gatekeeper	Storyteller	Required failure -> NOT_SURE/BLOCK; Storyteller -> template
Personal mode	Vault Keeper	Historian	No Vault result -> generic mode only when disclosed
Allergy, diet, health code	Medic	None	No Medic result -> do not recommend
Declared goals	Coach	Historian	Omit goal/history dimension; never infer
Rejected or qualified	None	Bargain Hunter	Return without an alternative
Context invariant tenantId, userId, purpose, consent snapshot, allowlists, budgets, deadlines, and configuration versions are runtime authority. They are not prompt instructions.			

3. Conductor algorithm

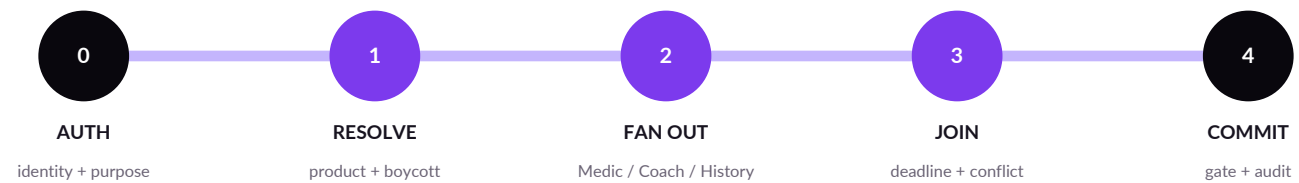


Figure 2. The request path resolves shared truth before private personalization, then joins all results under a fixed deadline.

3.1 Phase sequence

Phase	Action	Synchronization rule
0. Authenticate	Resolve user, tenant, purpose, limits	Reject mismatches before any agent or tool
1. Identify	Resolve exact product and boycott evidence	Do not start private fit on ambiguous identity
2. Consent	Snapshot permitted personal purposes	Bind all private reads to snapshot ID
3. Fan out	Run Medic, Coach, optional Historian independently	Separate budgets; no role-to-role calls
4. Join	Collect settled results until join deadline	Required failure is safe; optional failure reduces coverage
5. Finalize	Apply policy, confidence, gate, commit	One transaction and one final writer

3.2 TypeScript orchestration skeleton

```

const ctx = await createExecutionContext(request)
await bodyguard.validate(request, ctx)
const product = await identity.resolve(scanValue, ctx)
const boycott = await boycottResolution.resolve(product, ctx)
if (boycottTriggersStrictVeto(boycott, ctx))
  return gateAndCommit(buildBoycottDecision(product, boycott), ctx)

const consent = await consentService.snapshot(ctx)
const profile = await profileService.readFeatureCodes(consent, ctx)
const settled = await runSpecialistsWithDeadline(ctx, product, profile)
const joined = joinResults(settled, ctx)
return gateAndCommit(joined, ctx)

```

Deadline rule

The join deadline must be lower than the Elysia request timeout. Cancel late optional work first; never drop Medic, Referee, Skeptic, or Gatekeeper to save time.

4. Parallel execution and tool authorization

4.1 Deadline-aware fan-out

```
async function runSpecialistsWithDeadline(ctx, product, profile) {
  const controller = new AbortController()
  const remaining = Math.max(0, ctx.deadlineAt - Date.now())
  const timer = setTimeout(() => controller.abort('join_deadline'), remaining)
  try {
    return await Promise.allSettled([
      runMedic(ctx, product, profile, controller.signal),
      runCoach(ctx, product, profile, controller.signal),
      runHistorian(ctx, product, profile, controller.signal),
    ])
  } finally { clearTimeout(timer) }
}
```

Use `Promise.allSettled`, not `Promise.all`, because one optional failure must not erase valid results. Normalize every rejection into a typed failure before the join stage.

4.2 Seven checks before a tool call

Check	Question	Failure
Need	Is this tool necessary for the typed task?	Reuse existing result or deny
Allowlist	Is toolId allowed for this role?	Deny and audit
Identity	Does context match authenticated user and tenant?	Block
Purpose/consent	Is the private read currently permitted?	Generic mode or block
Minimization	Are only allowed feature codes requested?	Reject excess fields
Budget/deadline	Are calls, cost, and time still available?	Approved fallback
Schema	Do request and response pass Zod validation?	Normalize failure

4.3 Tool wrapper boundary

```
authorizeToolCall(role, toolId, ctx, input)
reserveBudget(ctx, role, toolId)
const result = await serviceAdapter(input, ctx)
const output = outputSchema.parse(result)
appendToolAudit({ requestId: ctx.requestId, role, toolId, outcome })
return output
```

5. Join, conflict, and confidence

5.1 Join algorithm

- Convert rejected promises and timeouts into typed `AgentResult` failures.
- Verify that each result belongs to the current request, role, and configuration versions.
- Reject unknown fields and evidence references that do not exist or are outside the allowed scope.
- For a missing required result, choose its documented `NOT_SURE` or `BLOCKED` outcome.
- For a missing optional result, remove the dimension and lower coverage; never score it as zero or neutral.
- Normalize claims by subject and predicate, then preserve supporting and opposing positions.
- Run Referee, then Skeptic, then Gatekeeper. Do not let the explanation model participate in these gates.

5.2 Conflict resolution

Conflict	Referee action	Decision effect
Boycat says boycott; internal source disagrees	Persist both positions and policy resolution	needs_review or policy-specific qualified result
Coach likes product; Medic finds allergy	Hard safety gate wins	not_recommended with warning
Historian unavailable	Omit history dimension	Lower coverage; no neutral score
Storyteller invents a benefit	Gatekeeper rejects explanation	Use deterministic template
Consent changed before commit	Recheck and invalidate private draft	Generic result or blocked commit

5.3 Confidence is not agent voting

Signal	Raises confidence	Lowers confidence
Source tier	Direct, validated source	Unknown or weak source
Freshness	Observed within policy window	Stale or missing timestamp
Coverage	All required dimensions ran	Required/optional dimensions missing
Directness	Exact product and claim match	Brand-level inference or ambiguity
Conflict	No material unresolved conflict	Competing credible positions

Important

Three agents agreeing does not create truth if they all depend on the same weak source. Confidence comes from evidence quality, freshness, directness, coverage, and unresolved conflict.

6. Gatekeeper and the final commit

6.1 Final validation order

Order	Gate	Reject when
1	Identity and tenancy	Request, result, user, or tenant do not match
2	Consent	Snapshot is missing, withdrawn, expired, or wrong purpose
3	Schema	Unknown fields, malformed status, or invalid enum
4	Evidence	Unsupported claim, missing reference, stale policy violation
5	Safety and boycott policy	Hard gate was skipped or overridden
6	Privacy	Explanation or logs expose raw private fields
7	Configuration	Results use incompatible active versions
8	Audit	Material decision cannot be durably audited

6.2 Atomic persistence goal

Where the tenant database supports it, commit the following records in one transaction and only then populate Redis:

- `ai_decision` - final outcome, dimensions, warnings, confidence, and versions.
- `ai_decision_evidence` - the exact evidence references used.
- `ai_decision_disagreement` - material supporting/opposing claims and resolution.
- `ai_audit_event` - plan, tools, gates, fallbacks, commit ID, and redacted timing metadata.

```
await tenantDb.transaction(async (tx) => {
  const decisionId = await decisionRepo.insert(tx, decision)
  await evidenceRepo.insertMany(tx, decisionId, evidence)
  await disagreementRepo.insertMany(tx, decisionId, conflicts)
  await auditRepo.append(tx, buildCommitAudit(ctx, decisionId))
})
await readyCache.put(buildCacheArtifact(decision))
```

Commit-before-cache

Redis is an optimization and coordination layer, never the source of truth. A failed cache write must not erase a committed decision; a cache hit must still pass consent, freshness, Skeptic, and Gatekeeper checks.

7. Mapping into the Zaatot backend

7.1 Proposed files

```
src/ai/
agent/product-fit/{product-fit.agent.ts, product-fit.prompt.agent.ts,
product-fit.schema.agent.ts}
agent/storyteller/{storyteller.agent.ts, storyteller.prompt.agent.ts,
storyteller.schema.agent.ts}
orchestration/{execution-context.ts, conductor.ts, join.ts, gatekeeper.ts}
tool/catalog/{catalog.tool.ts, catalog.dto.tool.ts}
tool/boycott/{boycott.tool.ts, boycott.dto.tool.ts}
tool/profile/{profile.tool.ts, profile.dto.tool.ts}
tool/product-fit/{product-fit.tool.ts, product-fit.dto.tool.ts}
src/module/main/{boycott-resolution, product-fit, alternative}/...
src/module/user/{profile, consent, ai-decision}/...
```

7.2 Service boundaries

Component	Owns	Must not own
service_product_identity	Barcode/product resolution and ambiguity	User fit or recommendation
service_boycott_resolution	Boycat + internal evidence and disagreement	Personal health or goals
service_consent/profile	Consent snapshot and minimized feature codes	Product scoring
service_product_safety	Deterministic hard restrictions	Free-form diagnosis
service_product_fit	Versioned dimension scoring	Final commit without gates
Conductor	Planning, budgets, deadlines, join	SQL or business truth
Gatekeeper	Final validation and commit	Inventing claims

7.3 HTTP surface

Method	Route	Purpose
PUT	/user/ai-profile	Update consented goals, restrictions, preferences
POST	/user/ai-consent	Grant or withdraw versioned purposes
POST	/ai/product-decision	Run synchronized identity -> boycott -> fit flow
GET	/ai/product-decision/:id	Read one authenticated user's decision
POST	/ai/product-decision/:id/feedback	Record feedback without rewriting history

Current context gap

Replace or extend the present contact-oriented AI tool context before personalization. A context that expects contact_phone or produces user_id: -1 is not acceptable for user-scoped decisions.

8. Failure behavior and tests

8.1 Safe failure matrix

Failure	Required behavior	Forbidden behavior
Product ambiguous	Ask user to select or rescan	Guess product
Boycott provider unavailable	Use compatible fresh evidence or unknown	Convert outage to not_boycotted
Ingredients missing for allergy user	NOT_SURE with safe next step	Assume no allergen
Consent withdrawn mid-request	Remove private dimensions and block private commit	Use earlier snapshot without recheck
Specialist timeout	Apply required/optional fallback	Wait beyond request deadline
Malformed model output	Reject and use template fallback	Return raw prose
Cross-tenant mismatch	Block, audit, preserve redacted evidence	Retry with broader credentials
Audit failure	Block material commit or mark unhealthy	Return unaudited complete decision

8.2 Minimum synchronization tests

- Required Medic timeout returns NOT_SURE or a configured block; optional Historian timeout only lowers coverage.
- One specialist rejection does not discard other fulfilled results.
- Late optional work is aborted and cannot mutate a committed decision.
- A role cannot call a tool outside its immutable allowlist.
- A tenant/user mismatch is blocked before the service adapter executes.
- Consent withdrawal between profile read and Gatekeeper prevents private commit.
- Conflicting boycott sources are persisted and reduce confidence.
- Storyteller invention is rejected and deterministic explanation is returned.
- Duplicate Idempotency-Key returns the same logical decision without duplicate records.
- Cache hit after evidence/config supersession is rejected and recomputed or returns NOT_SURE.

8.3 Observability without private leakage

Record request/trace IDs, role status, tool ID, duration, timeout, fallback code, evidence IDs, configuration versions, coverage, confidence band, and gate outcome. Never log raw profiles, health text, credentials, complete prompts, or provider secrets.

9. Implementation checklist

9.1 Build order

Order	Deliverable	Exit evidence
1	Approve outcome and boycott policies	Versioned enums, thresholds, and owner approval
2	Harden tenant migrations	Historical-schema tests and reversible migration plan
3	Create ExecutionContext and tool authorization	Cross-tenant and allowlist red-line tests
4	Create AgentResult, Claim, EvidenceRef, Warning schemas	Unknown fields rejected
5	Unify product and boycott resolution services	Uncertainty and disagreement preserved
6	Build deterministic Medic and Coach	Golden rule scenarios pass without an LLM
7	Implement Conductor fan-out and join deadline	Required/optional timeout tests pass
8	Implement Referee, Skeptic, Gatekeeper commit	Consent race, audit, and idempotency tests pass
9	Add authenticated product-decision route	One stable end-to-end vertical slice
10	Add tool-less Storyteller behind feature flag	Faithfulness evaluation and template fallback

9.2 Definition of done

- Every role has a purpose, required/optional classification, deadline, budget, allowlist, input/output schema, and safe fallback.
- All hard restrictions and final gates are deterministic and tested.
- The model can be disabled while product identity, boycott, safety, scoring, confidence, and commit still work.
- Every final claim maps to evidence; every conflict remains visible; every configuration is versioned.
- Only Gatekeeper can commit, and Redis is populated only after durable persistence.
- No request can cross user or tenant boundaries, and consent is rechecked immediately before commit.
- Timeout, retry, provider outage, malformed output, audit failure, and duplicate request paths have tests.

Recommended first implementation

Build one synchronous deterministic pipeline with Conductor, Detective, Boycott Investigator, Vault Keeper, Medic, Coach, Referee, Skeptic, and Gatekeeper. Add Historian, alternatives, workers, and Storyteller only after that vertical slice is safe and observable.

Release rule

FAST when evidence is strong. QUALIFIED when evidence is limited. NOT_SURE when required evidence is thin. BLOCKED when safety, privacy, consent, tenancy, schema, or audit integrity fails. AUDITED in every case.